

Escola Superior de Tecnologia e Gestão
Licenciatura em Engenharia Informática

Estrutura de Dados 2025/2026

Trabalho prático nº 1

Serviços de Reparação

Bernardo José Inácio Gomes a2025145298

Elaborado em: 18/04/2026

Índice

Introdução.....	3
Estrutura de Dados	4
Fluxo de Vida (Estados).....	5
Registo e Gestão de Ativos.....	5
Regras de Edição	5
Encerramento.....	6
Relatório Mensal	6
Lógica de Persistência e de Armazenamento	7
Leitura do Ficheiro Binário	7
Escrita do Ficheiro Binário	8
Regras de Bloqueio (Integridade)	9
Algoritmos de Ordenação	11
<i>Bubble Sort</i> (Ordenação Alfabética).....	11
Linear Insertion (Ordem Crescente de datas de conclusão)	12
Algoritmos de Pesquisa.....	13
Demonstração de Resultados e Análise	14
Teste 1:	14
Teste 2:	14
Teste 3:	15
Teste 4:	15
Teste 5:	16
Conclusão.....	17
Bibliografia	18

Introdução

Este trabalho tem como objetivo, criar uma aplicação tendo como base a linguagem C, que permite registar, listar, editar vários tipos de serviços para uma loja de reparações de informática.

Neste trabalho foram utilizados conceitos lecionados nas aulas de Estrutura de Dados tais como: *structs*, *arrays*, ficheiros de texto e ficheiros binários. Para uma melhor organização da informação foram aplicados algoritmos de ordenação como o *Bubble Sort* e o *Linear Insertion*.

Para o contexto da utilização do programa, as datas são colocadas pelo utilizador da seguinte maneira: **Dia Mês hora minuto segundo ano**.

Por exemplo para escrever a data 17/04/2026 às 09:46, no programa seria da seguinte maneira: **17 04 09 46 00 2026**.

Estrutura de Dados

O funcionamento de toda a aplicação tem como base uma **struct Serviço**, onde esta é constituída por vários campos de forma a gerir as informações ao longo de cada execução.

Sendo eles agrupados por:

- **Identificadores numéricos e de texto** - `codigoID`, `tipoServico`, `cliente`, `equipamento`...
No contexto da utilização da aplicação estes identificadores principalmente o `codigoID`, `tipoServico(string)` e `cliente(string)` servem para efetuar diferentes tipos de listagens. O `codigoID` é gerado de forma sequencial através da leitura do ficheiro binário, obtendo assim em cada execução de programa o último código ID maior.
- **Controlos de Estado** – Para um melhor fluxo da aplicação foram criadas 3 variáveis inteiras que servem como estados booleanos (1 – True, 0 – False), sendo elas `concluido`, `pago` e `arquivado`. A variável de `concluido` serve para registar se determinado serviço está ou não concluído, a variável de `pago` serve para registar se um serviço está ou não pago e por último a variável de `arquivado` serve para, registar quando um serviço é colocado em relatório mensal (ficheiro CSV).
- **Custos e Componentes** - Para tratar as informações relativas aos custos foi criada uma variável do tipo real (**float**) com o nome de `precoServico` esta variável permite registar o preço de cada serviço sendo que para os serviços de Diagnóstico e de Limpeza esta variável representa o preço final do serviço, mas para os serviços de Reparação e de Manutenção esta variável representa um valor que é pago por segundo. Para tratar as informações relativas aos componentes foi criado um *array* do tipo *float* com o nome de `precoComponentes`, uma variável do tipo inteiro que serve para registar o número de componentes que podem ser substituídos (valor máximo = 10) e ainda um campo de observações (**string**) para registar qual peça ou componente foi substituído durante a Reparação/Manutenção.
- **Struct Data e `time_t`** – Estas estruturas foram criadas de forma a auxiliar, o fluxo e o processamento dos dados na aplicação.
A struct Data serve essencialmente para obter os dados em formato de inteiros pela ordem descrita na introdução. Através da utilização do cabeçalho **time.h** é possível criar uma estrutura do tipo `time_t` que possibilita o tratamento dos dados provenientes da **struct Data** com a finalidade de auxiliar em cálculos de duração de serviços, ordenações cronológicas e ainda formatação do tempo em **string**.

Fluxo de Vida (Estados)

Este capítulo refere-se ao modo como a aplicação funciona, serão dados alguns exemplos de maneira que seja mais fácil o entendimento da aplicação.

Registo e Gestão de Ativos

Numa primeira instância o utilizador pode fazer registos de serviços, nesta opção o utilizador tem de escolher o tipo de serviço que quer, o nome do cliente, o nome do equipamento onde será realizado o serviço e quem irá ser o funcionário que irá realizar o serviço.

Devido à existência de um *array* de estrutura de serviços que podemos chamar de serviços ou `servicosAtivos` é possível registar diferentes serviços na mesma execução de programa. Este *array* tem um tamanho fixo de 100 sendo **possível** alterar este valor já que foi definido como constante no início da aplicação.

Regras de Edição

Quando um serviço não está concluído, é possível fazer algumas edições diferentes tais como: adicionar componentes caso o serviço seja de manutenção ou de reparação, alterar o preço do serviço, eliminar um serviço, ordenar por ordem alfabética.

Caso o serviço já esteja concluído não é possível fazer edições ou seja alterar o preço ou adicionar componentes. Também não é possível colocar uma data de conclusão anterior à data de início do registo.

A aplicação está desenhada para que quando o utilizador dê um serviço como pago esta funcionalidade só é possível se o serviço em questão estiver concluído.

Existem listagens feitas de maneiras diferentes que são possíveis de fazer na mesma execução, devido ao uso do *array* de ativos.

Encerramento

O utilizador deve sair sempre da aplicação selecionando a **opção 0**, para que os dados fiquem guardados no ficheiro binário de Serviços, e assim não haver dados inválidos ou corrompidos. Caso exista uma **interrupção abrupta** da aplicação, os dados podem não ficar guardados e gerar dados inválidos na próxima execução do programa.

Relatório Mensal

De maneira a simular o fim do mês existe na aplicação a opção de relatório mensal, este relatório é gerado a partir dos serviços concluídos pertencentes ao *array* de estruturas de ativos. Estes serviços concluídos são adicionados a um ficheiro do tipo CSV, posteriormente são copiados para um outro *array* do mesmo tipo da estrutura de ativos chamado de *servicosConcluidos*, no fim os serviços que foram adicionados ao ficheiro CSV são dados como arquivados (*arquivado* = 1) e são eliminados da estrutura de ativos. Ao fim de ser gerado o relatório mensal, é aconselhável ao utilizador sair do programa selecionando a opção 0.

Lógica de Persistência e de Armazenamento

Este capítulo irá tratar da maneira como os dados são guardados da maneira a serem preservados entre sessões.

Leitura do Ficheiro Binário

Na função main da aplicação a leitura do ficheiro binário é feita de maneira individual, ou seja, é lida estrutura a estrutura e depois é verificada a condição de serviço arquivado. Caso o serviço tenha a variável de arquivado a 1 significa que em alguma das sessões anteriores foi efetuado o relatório mensal, o que significa que aquando de uma nova sessão a estrutura de concluídos ou arquivados irá ser devidamente preenchida só com os serviços arquivados, por outro lado se a variável de arquivado seja zero então a estrutura de ativos ou de serviços será devidamente preenchida.

```
int main(){  
  
    FILE *f;  
    //Variaveis para opcao , codigo e ID  
    int opcao ,   codigo;  
    int ultimoIDGravado = obterMaiorId() + 1;  
    int totalServicos = 0 , totalConcluidos = 0;  
    Servico servicos[TAMANHO];  
    Servico servicosConcluidos[TAMANHO];  
  
    f = fopen("Servicos.dat" , "rb");  
    if(f != NULL){  
  
        Servico tmp;  
        while (fread(&tmp , sizeof(Servico) , 1 , f) == 1)  
        {  
            if(tmp.arquivado == 0){  
                servicos[totalServicos++] = tmp;  
            }  
            else{  
                servicosConcluidos[totalConcluidos++] = tmp;  
            }  
        }  
        fclose(f);  
    }  
}
```

Figura 1 - Leitura no Main do Ficheiro Binário

Como se pode observar na figura 1 a aplicação está a ler todo o ficheiro binário através de **while (fread (...))** permitindo assim que as estruturas tanto de serviços como de concluídos se mantenham entre sessões.

Escrita do Ficheiro Binário

Para a escrita do ficheiro binário, foi utilizado um subprograma próprio que permite a escrita da estrutura de ativos de uma só vez seguida caso exista da escrita da estrutura de concluídos.

Este subprograma é chamado antes do utilizador sair do programa, de maneira a escrever sempre as alterações efetuadas ao longo da sessão.

```
//Sub Programa para reescrever no ficheiro
void guardarTabelaUnica(Servico ativos[], int totalAtivos , Servico concluidos[], int
totalConcluidos){
    FILE *f;
    f = fopen("Servicos.dat" , "wb");
    if(f != NULL){
        fwrite(ativos , sizeof(Servico) , totalAtivos , f);
        fwrite(concluidos , sizeof(Servico) , totalConcluidos , f);

        fclose(f);
    }
}
```

Figura 2-Subprograma para escrita no ficheiro binário

```
    }

    printf("Selecione 1 para continuar 0 para Sair\n");
    scanf("%d" , &opcao);
    getchar();

}while(opcao != 0);
    guardarTabelaUnica(servicos , totalServicos , servicosConcluidos , totalConcluidos);
}
```

Figura 3-Chamada do subprograma para guardar os dados entre sessões

Regras de Bloqueio (Integridade)

Este capítulo irá se focar nas *flags* de concluído e de pago, que servem para registar o estado do serviço consoante as sessões da aplicação.

Um serviço que seja dado por concluído o seu preço não pode ser alterado nem a edição de componentes ou seja não é possível adicionar componentes a um serviço depois de este estar concluído.

Só é possível obter o preço de serviços como Reparação e Manutenção quando o serviço estiver concluído já que o preço destes depende do tempo que levou a concluir o serviço. Serão sempre possíveis algumas listagens como por exemplo: listar serviços por cliente, listar serviços concluídos por ano, listar serviços concluídos por tipo...

Para o serviço ser dado como concluído tem de ser feita a seguinte verificação: a data colocada pelo utilizador no formato descrito acima terá sempre de ser maior do que a data do início de registo. Para isso foi utilizada a função *difftime* que pertence à biblioteca *time.h* e que permite assim obter a diferença em segundos de duas estruturas do tipo *time_t*.

```
//Logica das datas
for(int i = 0 ; i < total ; i++){

    if(servicos[i].codigoID == codigo){
        if(servicos[i].concluido == 1){
            printf("Valor ja atribuido!\n");
        }

        else{
            printf("Insira a data de fim no formato de inteiros: (Dia Mes hh mm ss 2026)
            \n");
            scanf("%d %d %d %d %d %d" , &servicos[i].dataDeFim.dia , &servicos[i].
            dataDeFim.mes , &servicos[i].dataDeFim.hora , &servicos[i].dataDeFim.minuto , &
            servicos[i].dataDeFim.segundo , &servicos[i].dataDeFim.ano);
            getchar();

            time_t ti = construirEstrutura(servicos[i].dataDeInicio.hora , servicos[i].
            dataDeInicio.minuto , servicos[i].dataDeInicio.segundo , servicos[i].
            dataDeInicio.dia , servicos[i].dataDeInicio.mes , servicos[i].dataDeInicio.
            ano);
            time_t tf = construirEstrutura(servicos[i].dataDeFim.hora , servicos[i].
            dataDeFim.minuto , servicos[i].dataDeFim.segundo , servicos[i].dataDeFim.dia ,
            servicos[i].dataDeFim.mes , servicos[i].dataDeFim.ano);

            diferenca = difftime(tf , ti);

            if(diferenca > 0){
                servicos[i].concluido = 1;
                printf("Servico Concluido Com Sucesso!\n");
            }
            else{
                printf("Coloque a data de fim superior a data de inicio\n");
            }
        }
    }
}
```

Figura 4-Verificação da data através da função *difftime*

O código representado na figura 4 pertence a um subprograma do tipo *void* chamado *servicoConcluido*.

Qualquer tipo de serviço para ser pago tem de estar concluído, para além disso o utilizador terá de colocar uma data correspondente à data de pagamento, sendo que esta não poderá ser inferior à data de conclusão.

```
for(int i = 0 ; i < total ; i++){
    if(servicos[i].codigoID == codigo && servicos[i].concluido == 1 && servicos[i].pago == 0){

        encontrou = 1;
        printf("Coloque a data de pagamento no formato de inteiros (Dia Mes hh mm ss 2026)\n");

        scanf("%d %d %d %d %d %d" , &dia , &mes , &hora , &minuto , &segundo , &ano);
        dataPagamento = construirEstrutura(hora , minuto , segundo , dia , mes , ano);

        fim = construirEstrutura(servicos[i].dataDeFim.hora , servicos[i].dataDeFim.minuto ,
                                servicos[i].dataDeFim.segundo , servicos[i].dataDeFim.dia , servicos[i].dataDeFim.mes , servicos[i].dataDeFim.ano);

        segundos = difftime(dataPagamento , fim);

        if(segundos >= 0){
            servicos[i].pago = 1;
            printf("O servico: %s com o codigo: %d , foi dado como pago\n" , servicos[i].ti
                  servicos[i].dataDePagamento.dia = dia;
                  servicos[i].dataDePagamento.mes = mes;
                  servicos[i].dataDePagamento.hora = hora;
                  servicos[i].dataDePagamento.minuto = minuto;
                  servicos[i].dataDePagamento.segundo = segundo;
                  servicos[i].dataDePagamento.ano = ano;
            }
        else{
            printf("Erro a data de pagamento nao pode ser anterior a data de conclusao\n");
        }
    }
}
```

Figura 5-Verificação das datas para registar um serviço como pago.

O excerto do código da figura 5 pertence a um subprograma do tipo *void* chamado, chamado *servicoPago*.

Algoritmos de Ordenação

Este capítulo é referente ao modo como foram implementados dois algoritmos de ordenação, um deles para ordenar alfabeticamente uma estrutura e o segundo para ordenar por ordem crescente as datas que de conclusão.

Bubble Sort (Ordenação Alfabética)

Para ordenar alfabeticamente os clientes que estão na estrutura de ativos/serviços foi utilizado o algoritmo bubble sort.

Este algoritmo é um dos algoritmos mais simples de ordenação e permite trocar elementos num dado *array* caso estes não estejam na ordem correta.

Este algoritmo consiste em iterar pelo *array* várias vezes de maneira em que a cada iteração o elemento, neste caso o maior, irá ocupar a última posição.

Como a ordenação é alfabética, neste caso, a condição para comparar os elementos seguintes foi feita recorrendo a ***strcasecmp*** que é uma função pertencente à biblioteca *string.h* e que permite comparar *strings* sem fazer distinção entre caracteres minúsculos ou maiúsculos.

```
//Ordem Alfabetica - Bubble Sort
void ordemAlfabetica(Servico servicos[] , int total){
    Servico aux;
    for(int i = 0 ; i < total - 1 ; i++){
        for(int j = 0 ; j < total-i-1 ; j++){
            if(strcasecmp(servicos[j].cliente , servicos[j + 1].cliente) > 0){
                aux = servicos[j];
                servicos[j] = servicos[j + 1];
                servicos[j + 1] = aux;
            }
        }
    }
}
```

Figura 6-Algoritmo de Bubble Sort

Linear Insertion (Ordem Crescente de datas de conclusão)

Para ordenar cronologicamente as datas de conclusão de cada serviço foi utilizado *array* da estrutura de serviços/ativos.

Este algoritmo consiste em iterar por cada elemento no *array* inserindo cada elemento de maneira a criar dois grupos: um grupo ordenado e outro não ordenado.

Começando pelo segundo elemento do *array* e comparando este elemento com o anterior, caso o elemento anterior seja maior que o segundo elemento então irá se realizar uma troca, e assim sucessivamente até os elementos estarem ordenados.

Para este caso como se pretende ordenar datas recorreu-se à estrutura do tipo *time_t* para facilitar a comparação.

```
//Ordem Crescente das datas de conclusao (Algoritmo Insertion Linear) // Servicos Ativos
void ordemCrescenteDatas(Servico servicos[], int total){
    time_t tempoAuxiliar , tempoAuxiliarDois;
    Servico aux , auxdois;
    int j;

    for(int i = 1 ; i <= total - 1 ; i++){
        aux = servicos[i];
        auxdois = servicos[i - 1];

        tempoAuxiliar = construirEstrutura(aux.dataDeFim.hora , aux.dataDeFim.minuto ,
        aux.dataDeFim.segundo , aux.dataDeFim.dia , aux.dataDeFim.mes , aux.dataDeFim.
        ano);

        tempoAuxiliarDois = construirEstrutura(auxdois.dataDeFim.hora , auxdois.
        dataDeFim.minuto , auxdois.dataDeFim.segundo , auxdois.dataDeFim.dia , auxdois.
        dataDeFim.mes , auxdois.dataDeFim.ano);

        j = i;

        while (j > 0 && tempoAuxiliarDois > tempoAuxiliar)
        {
            servicos[j] = servicos[j - 1];
            j = j-1;
        }
        servicos[j] = aux;
    }

    printf("Datas ordenadas com sucesso!\n");
}
```

Figura 7-Algoritmo Linear Insertion

Algoritmos de Pesquisa

A localização de registos para edição, conclusão ou pagamento é feita através do uso do algoritmo de pesquisa de *linear search*.

O funcionamento deste algoritmo consiste em iterar por todos os elementos do *array* até encontrar o elemento pretendido.

```
//Sub Programa Para Listar os Servicos a decorrer
void listarServicos_decorrer(Servico servicos[] , int total){
    char servicoProcurado[100];

    printf("Digite o tipo de servico que deseja procurar (Reparacao , Diagnostico , Limpeza ,
    Manutencao): \n");
    fgets(servicoProcurado , 100 , stdin);
    removerEnter(servicoProcurado);

    for(int i = 0 ; i < total ; i++){
        if(strcmp(servicoProcurado , servicos[i].tipoServico) == 0 && servicos[i].concluido == 0){
            printf("Codigo: %d | Cliente: %s | Equipamento: %s \n" , servicos[i].codigoID ,
                servicos[i].cliente , servicos[i].equipamento);
        }
    }
}
```

Figura 8-Exemplo do algoritmo de *linear search*

No exemplo acima pretende-se pesquisar pelo tipo de serviço comparando a *string* introduzida pelo utilizador com a *string* pertencente à estrutura de serviços e também se o serviço está concluído.

Demonstração de Resultados e Análise

Este capítulo destina-se à demonstração de alguns testes feitos na aplicação, a realização destes testes foi feita com base na utilização de alguns scripts que estão disponíveis nos arquivos do trabalho, para efetuar a execução de cada script foi utilizado o *powerShell* do vs Code. Entre cada execução de scripts é aconselhável eliminar o ficheiro “Servicos.dat”, já que em cada script a pesquisa pelo código ID começa sempre por um.

Teste 1:

```
17 - Duracao Maxima , Minima e Media por Tipo de Servico
18 - Dar Servico Pago
Selecione o tipo de Servico: 1 - Reparacao      2 - Diagnostico      3 - Manutencao 4-Limpeza
Insira a data de inicio no formato de inteiros: (Dia Mes hh mm ss 2026)
Insira o nome do cliente:
Insira o nome do equipamento:
Introduza o nome do funcionario responsavel:
Selecione 1 para continuar 0 para Sair
Selecione o tipo de opcao : 2 - Registo 3 - Servico Concluido  4 - Relatorio Mensal  5 - Apresentar
6 - Eliminar Um servico 7-Editar Componentes  8-Preco Total do Servico  9-Listar Servicos Por Cliente
10 - Listar Servicos Concluidos Por Ano  11 - Atualizar preco de servico  12 - Listar Servicos a decorrer
13 - Listar Servicos Concluidos Por Tipo  14 - Ordenar Clientes Por Ordem Alfabetica
15 - Ordem Crescente das Datas de Conclusão  16 - Duracao Minima , Maxima e Media
17 - Duracao Maxima , Minima e Media por Tipo de Servico
18 - Dar Servico Pago
Introduza o codigo:
Insira a data de fim no formato de inteiros: (Dia Mes hh mm ss 2026)
Servico Concluido Com Sucesso!
Selecione 1 para continuar 0 para Sair
Selecione o tipo de opcao : 2 - Registo 3 - Servico Concluido  4 - Relatorio Mensal  5 - Apresentar
6 - Eliminar Um servico 7-Editar Componentes  8-Preco Total do Servico  9-Listar Servicos Por Cliente
10 - Listar Servicos Concluidos Por Ano  11 - Atualizar preco de servico  12 - Listar Servicos a decorrer
13 - Listar Servicos Concluidos Por Tipo  14 - Ordenar Clientes Por Ordem Alfabetica
15 - Ordem Crescente das Datas de Conclusão  16 - Duracao Minima , Maxima e Media
17 - Duracao Maxima , Minima e Media por Tipo de Servico
18 - Dar Servico Pago
Introduza o codigo:
Coloque a data de pagamento no formato de inteiros (Dia Mes hh mm ss 2026)
O servico: Reparacao com o codigo: 1, foi dado como pago
Selecione 1 para continuar 0 para Sair
```

Figura 9-Teste 1- Registo, Conclusão e Pagamento

Este teste foi realizado para demonstrar um ciclo de registo, conclusão e dar um serviço como pago através do código id, o script utilizado para a realização deste teste tem como nome “teste_sucesso.txt”.

Teste 2:

```
Insira o nome do equipamento:
Introduza o nome do funcionario responsavel:
Selecione 1 para continuar 0 para Sair
Selecione o tipo de opcao : 2 - Registo 3 - Servico Concluido  4 - Relatorio Mensal  5 - Apresentar
6 - Eliminar Um servico 7-Editar Componentes  8-Preco Total do Servico  9-Listar Servicos Por Cliente
10 - Listar Servicos Concluidos Por Ano  11 - Atualizar preco de servico  12 - Listar Servicos a decorrer
13 - Listar Servicos Concluidos Por Tipo  14 - Ordenar Clientes Por Ordem Alfabetica
15 - Ordem Crescente das Datas de Conclusão  16 - Duracao Minima , Maxima e Media
17 - Duracao Maxima , Minima e Media por Tipo de Servico
18 - Dar Servico Pago
Introduza o codigo:
Insira a data de fim no formato de inteiros: (Dia Mes hh mm ss 2026)
Servico Concluido Com Sucesso!
Selecione 1 para continuar 0 para Sair
Selecione o tipo de opcao : 2 - Registo 3 - Servico Concluido  4 - Relatorio Mensal  5 - Apresentar
6 - Eliminar Um servico 7-Editar Componentes  8-Preco Total do Servico  9-Listar Servicos Por Cliente
10 - Listar Servicos Concluidos Por Ano  11 - Atualizar preco de servico  12 - Listar Servicos a decorrer
13 - Listar Servicos Concluidos Por Tipo  14 - Ordenar Clientes Por Ordem Alfabetica
15 - Ordem Crescente das Datas de Conclusão  16 - Duracao Minima , Maxima e Media
17 - Duracao Maxima , Minima e Media por Tipo de Servico
18 - Dar Servico Pago
Introduza o codigo:
Coloque a data de pagamento no formato de inteiros (Dia Mes hh mm ss 2026)
Erro a data de pagamento nao pode ser anterior a data de conclusao
Selecione 1 para continuar 0 para Sair
PS C:\Users\berna\Desktop\11ET\Primeiro Ano Segundo Semestre\Estrutura de Dados\Trabalho 1>
```

Figura 10-Teste 2, erro ao tentar colocar uma data de pagamento anterior à data de conclusão

Este teste tem o intuito de demonstrar uma das lógicas de bloqueio, em que o utilizador regista primeiramente um serviço de seguida conclui esse serviço e ao dar o serviço como pago o utilizador coloca uma data anterior à data de conclusão, obtendo assim uma mensagem de erro. O script utilizado para este teste tem como nome: “teste_erro_data.txt”.

Teste 3:

```

Insira o nome do cliente:
Insira o nome do equipamento:
Introduza o nome do funcionario responsavel:
Selecione 1 para continuar 0 para Sair
Selecione o tipo de opcao : 2 - Registo 3 - Servico Concluido 4 - Relatorio Mensal 5 - Apresentar
6 - Eliminar Um servico 7-Editar Componentes 8-Preco Total do Servico 9-Listar Servicos Por Cliente
10 - Listar Servicos Concluidos Por Ano 11 - Atualizar preco de servico 12 - Listar Servicos a decorrer
13 - Listar Servicos Concluidos Por Tipo 14 - Ordenar Clientes Por Ordem Alfabetica
15 - Ordem Crescente das Datas de Conclusão 16 - Duracao Minima , Maxima e Media
17 - Duracao Maxima , Minima e Media por Tipo de Servico
18 - Dar Servico Pago
Introduza o codigo:
Insira a data de fim no formato de inteiros: (Dia Mes hh mm ss 2026)
Servico Concluido Com Sucesso!
Selecione 1 para continuar 0 para Sair
Selecione o tipo de opcao : 2 - Registo 3 - Servico Concluido 4 - Relatorio Mensal 5 - Apresentar
6 - Eliminar Um servico 7-Editar Componentes 8-Preco Total do Servico 9-Listar Servicos Por Cliente
10 - Listar Servicos Concluidos Por Ano 11 - Atualizar preco de servico 12 - Listar Servicos a decorrer
13 - Listar Servicos Concluidos Por Tipo 14 - Ordenar Clientes Por Ordem Alfabetica
15 - Ordem Crescente das Datas de Conclusão 16 - Duracao Minima , Maxima e Media
17 - Duracao Maxima , Minima e Media por Tipo de Servico
18 - Dar Servico Pago
Insira o tipo de servico:
Sucesso!
Selecione 1 para continuar 0 para Sair
PS C:\Users\berna\Desktop\LEI\Primeiro Ano Segundo Semestre\Estrutura de Dados\Trabalho 1>

```

Figura 11-Teste 3- Gerar o ficheiro CSV (Relatório Mensal)

Neste teste pretende-se simular o fim do mês de maneira a gerar o ficheiro em formato de CSV, para a realização deste teste foram registados dois clientes de seguida os serviços foram dados como concluídos e após a conclusão destes serviços foi gerado o CSV selecionando a opção 4.

O script utilizado neste teste tem como nome: “teste_gera_csv.txt”.

Teste 4:

```

Estrutura Ativos:
Cliente: Ana Cliente
Codigo: 5
Tipo de Servico: Reparacao
Concluidos: 0
Pago: 0
Observacoes:
Cliente: Carlos Cliente
Codigo: 4
Tipo de Servico: Reparacao
Concluidos: 0
Pago: 0
Observacoes:
Cliente: Diana Cliente
Codigo: 3
Tipo de Servico: Reparacao
Concluidos: 0
Pago: 0
Observacoes:
Cliente: Filipe Cliente
Codigo: 2
Tipo de Servico: Reparacao
Concluidos: 0
Pago: 0
Observacoes:
Cliente: Joao Cliente
Codigo: 1
Tipo de Servico: Reparacao
Concluidos: 0
Pago: 0
Observacoes:

```

Figura 12-Teste 4, ordenação alfabética (Bubble Sort)

O objetivo deste teste é demonstrar o algoritmo de ordenação alfabética de bubble sort, onde foram registados cinco clientes. Inicialmente o primeiro cliente que foi registado foi o João de seguida foi o Filipe e assim sucessivamente, após a seleção da opção 14 foi possível ordenar alfabeticamente os clientes dentro do próprio *array* da estrutura de serviços como demonstrado na figura 12. O nome do script utilizado foi: “teste_ordenacao_alfabetica.txt”.

Teste 5:

```
Estrutura Ativos:  
Cliente: Ana Cliente  
Codigo: 5  
Tipo de Serviço: Reparacao  
Concluidos: 1  
Pago: 0  
Observacoes:  
Cliente: Carlos Cliente  
Codigo: 4  
Tipo de Serviço: Reparacao  
Concluidos: 1  
Pago: 0  
Observacoes:  
Cliente: Diana Cliente  
Codigo: 3  
Tipo de Serviço: Reparacao  
Concluidos: 1  
Pago: 0  
Observacoes:  
Cliente: Filipe Cliente  
Codigo: 2  
Tipo de Serviço: Reparacao  
Concluidos: 1  
Pago: 0  
Observacoes:  
Cliente: Joao Cliente  
Codigo: 1  
Tipo de Serviço: Reparacao  
Concluidos: 1  
Pago: 0  
Observacoes:
```

Figura 13-Ordenação De Datas (Linear Insertion)

Com este teste pretende-se demonstrar a ordenação crescente das datas de conclusão utilizando o algoritmo de *linear insertion*. Para a realização deste teste foram registados 5 utilizadores e foi realizada a conclusão de cada serviço, mas com datas de conclusão diferentes sendo que para o primeiro cliente a duração seria de 5 horas, para o segundo a duração seria de 4 horas e assim sucessivamente, após a seleção da opção 15 o resultado será o que está representado na figura 13. O script utilizado tem como nome: "teste_ordenacao_datas.txt".

Conclusão

O desenvolvimento desta aplicação permitiu uma melhor consolidação de alguns conceitos lecionados na disciplina de Estrutura de Dados tais como: estruturas, *arrays*, leitura e escrita tanto de ficheiros binários como de texto (csv).

Para além da consolidação destes conceitos a realização deste trabalho permitiu um melhor entendimento sobre algoritmos de pesquisa e de ordenação nomeadamente: pesquisa linear, *bubble sort* e *linear insertion*.

Durante o desenvolvimento da aplicação foi necessário recorrer a uma nova biblioteca(*time.h*) em C para poder fazer a manipulação das datas inseridas pelo utilizador. Face a algumas adversidades durante o desenvolvimento da aplicação, foi utilizada inteligência artificial com a finalidade de auxiliar a deteção de alguns erros de lógica.

Bibliografia

Referências Bibliográficas:

Prompts feitos no Gemini (IA):

- “Como ler um ficheiro binário estrutura a estrutura e, durante a leitura, distribuir os dados entre dois arrays diferentes (ativos e arquivados) com base num campo booleano da *struct*?”
- “Explica a lógica para criar um relatório mensal que filtre serviços por tipo, os escreva num ficheiro CSV, os mova para um *array* de histórico e os remova do array principal de ativos.”
- “Como posso criar scripts de teste em ficheiros .txt para simular a entrada de um utilizador num programa interativo em C e executá-los através do terminal do VS Code (PowerShell)?”

[Bubble Sort - GeeksforGeeks](#)

[Insertion Sort Algorithm - GeeksforGeeks](#)

[strcasecmp \(\)- Comparar sequências sem distinção entre maiúsculas e minúsculas - Documentação da IBM](#)

[C time \(time.h\) Library - Date and Time Reference](#)